Freight Rate Prediction: Approach, Validation and Results

Assessment: Spotter, Machine Learning Engineer

Target: posted_rate (the rate quoted for a load)

Development data: 48,000 labeled loads, Jan to Oct 2025

Final scoring: 12,000 unlabeled loads (Nov to Dec 2025) and a fixed 31-day December chart

Final model: LightGBM, tuned with Optuna, trained on a log1p target

1. Summary

- **Final model:** LightGBM on a log1p-transformed target, tuned with Optuna, refit on all 48,000 labeled loads.
- **Validation:** time-based split. The last 31 days of training data (Oct 2025) were held out, because the real test set is a future period.
- **Holdout result:** RMSE 648.67, R^2 0.820, versus the best statistical baseline (rate per mile by equipment) at RMSE 683.87, R^2 0.800.
- **Main data fix:** about 0.6% of weights were negative. They were flipped to positive after checking that their distribution mirrors the positive weights.

2. Exploratory data analysis

2.1 Shape, nulls and cardinality

Item	Train	Validation
Rows	48,000	12,000
Date range	Jan 1 to Oct 31 2025	Nov 1 to Dec 31 2025
Missing weight	300	165
Missing market_index	374	249
Unique pickup cities	64	72
Equipment types	3 (Dry Van, Reefer, Flatbed)	3

Right away I noticed two things. The validation set has 8 more cities than training (72 vs 64), and its minimum latitude (25.5) is below anything in training (28.36). Because of this, I decided not to use city names as features. The model works from coordinates, distance and geometry, which carry over to cities it has never seen.

2.2 Negative weights (data-quality issue)

While checking the summary statistics I noticed negative weights in both train and test, with a minimum of -47,500. A truck cannot carry a negative load. I wrote the line below and a note in the notebook to check this (292 rows in train):

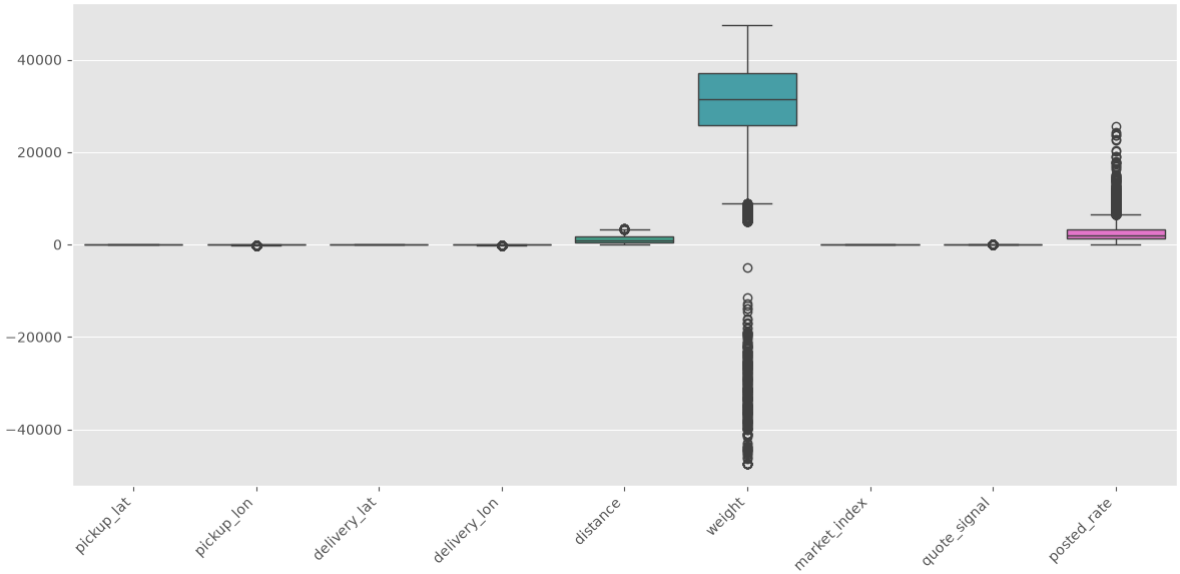
```
train[train.weight < 0].weight.describe()   # the negative rows
train[train.weight > 0].weight.describe()   # the positive rows for comparison
```

Then I compared the two groups. The negative rows looked like a mirror image of the positive ones:

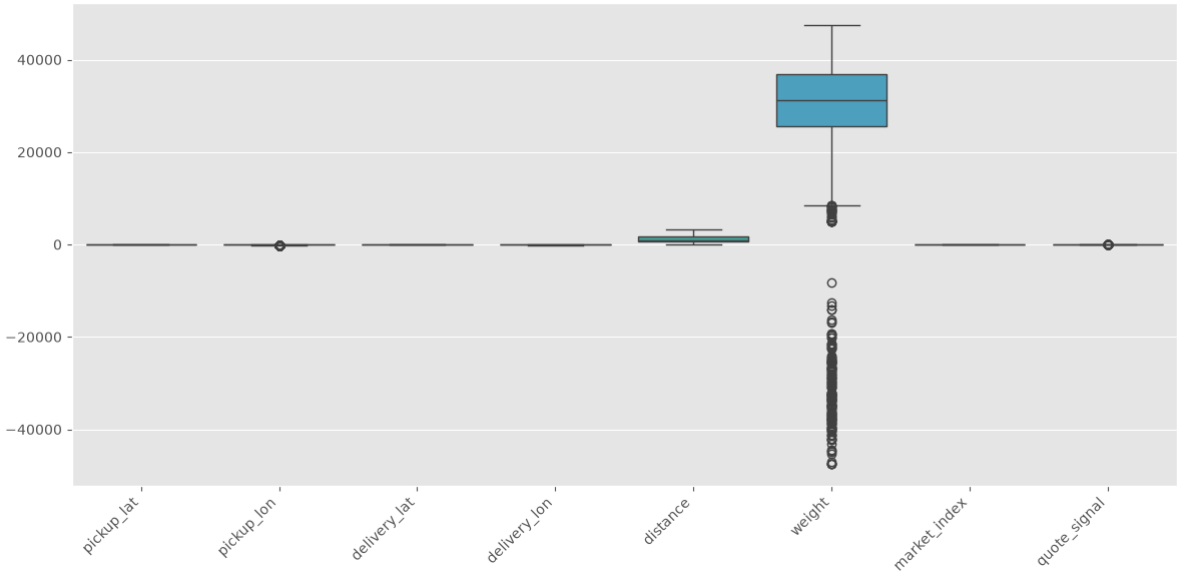
	Count	Mean	Std	Min	Max
Negative	292	-31,724	8,263	-47,500	-5,000
Positive	47,408	31,415	7,995	5,000	47,500

The magnitudes cover the same 5,000 to 47,500 range with a similar mean and spread, and the boxplots show the same thing. My conclusion was that this is a sign-entry error and not a different kind of load, so I flipped the sign in both train and test. Dropping the rows would have wasted data and left negatives in the unlabeled set anyway. This is the exact line I used:

```
train["weight"] = np.where(train.weight < 0, train.weight * -1, train.weight)
test["weight"]  = np.where(test.weight  < 0, test.weight  * -1, test.weight)
```



Boxplot of train columns: weight extends below zero.



Boxplot of validation columns: the same negative weights appear.

2.3 Missing values

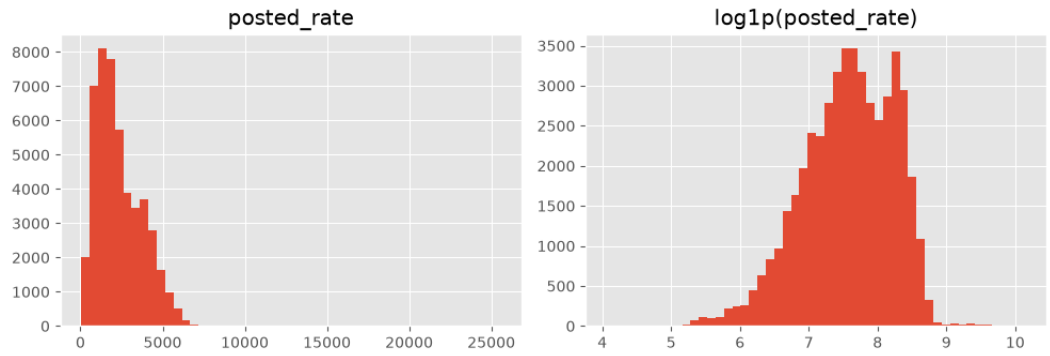
Next I looked at missing values: weight (300 rows) and market_index (374 rows). I filled weight with the median per equipment type, calculated on the training split only, so nothing leaks from the holdout or test data. market_index is filled with the training median inside the feature pipeline.

```
equip_median_wt = Xtrain.groupby("equipment")["weight"].median().to_dict()
# {"Dry Van": 31438.0, "Flatbed": 31538.0, "Reefer": 31586.0}
Xtrain["weight"] = Xtrain["weight"].fillna(Xtrain["equipment"].map(equip_median_wt))
```

2.4 Target

I then checked the target. posted_rate is right-skewed (skew 1.90), and after log1p the skew drops to -0.49. So I decided to model the target in log space and convert predictions back with expm1, using TransformedTargetRegressor.

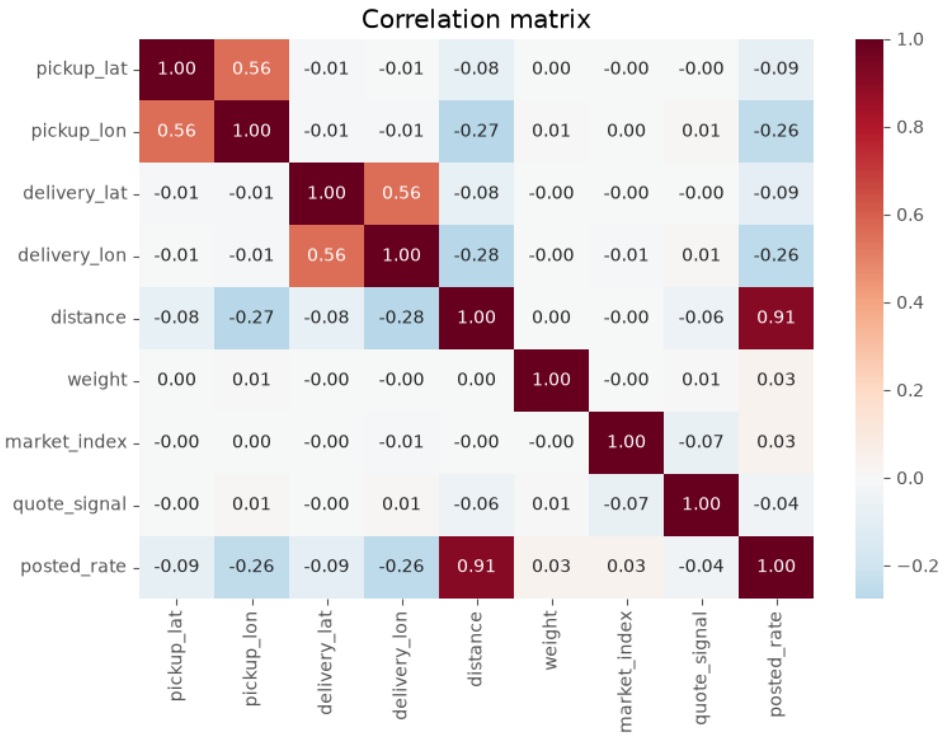
```
np.log1p(train["posted_rate"]).skew()   # -0.49 vs 1.90 before
```



posted_rate before and after log1p.

2.5 Correlations

From the correlation matrix I noticed that the features are not strongly correlated with each other. The only pair above 0.85 is distance and posted_rate (r = 0.91), which makes sense since price scales with miles. Weight, market_index and quote_signal show almost no linear correlation with the rate, so whatever they add must come from interactions or non-linear effects.



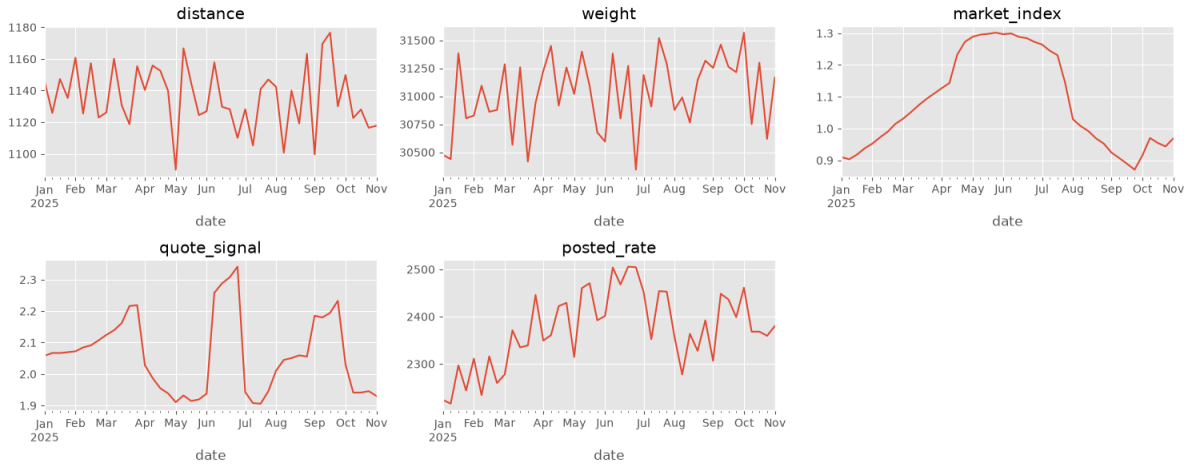
Correlation matrix.

2.6 Drift over time

Finally I plotted each numeric column over time. There are many loads per day, so I used the **weekly mean** to smooth the noise. I used the mean and not the median because the plots are only for spotting trends, and the columns have no extreme outliers once weight is fixed. The code is the notebook function plot_numeric_over_time, whose core is:

```
d = df.set_index("date").sort_index()
d = d[cols].resample("W").mean()   # weekly mean
```

I noticed that market_index in the validation period (mean 0.93) is lower than in training (mean 1.08), so the market level is drifting. This is why I chose a time-based split. A random split would hide the drift and give optimistic scores.



Weekly means of numeric columns, train.



Weekly means of numeric columns, validation.

3. Cleaning, split and feature engineering

3.1 Split strategy

The labeled data ends on Oct 31 2025 and the scored data covers the next two months. To copy this situation, I held out the last 31 days (October) as my validation set and trained on everything before it. Every step that learns from data (imputation medians, encoders, the model) is fitted on the training split only.

```
cutoff = X.date.max() - timedelta(days=31)      # 2025-09-30
Xtrain = X.query("date <= @cutoff")
Xval   = X.query("date >  @cutoff")
```

For tuning, I ran `TimeSeriesSplit(n_splits=5, test_size=31)` inside the training split with rows sorted by date. Each fold trains on the past and validates on the next 31 days. The October holdout stays untouched until the final check.

3.2 Features (feature-engine pipeline)

- **Date:** month, day of week and day of year, each also encoded as sin/cos so the calendar wraps around correctly.
- **Distance transforms:** log, sqrt, cube root, square, inverse and a distance bin. These give simple models a way to fit a curved relationship.
- **Geometry:** haversine miles between pickup and delivery, the ratio of road distance to haversine distance, and compass bearing.
- **Equipment:** ordinal-encoded. **Dropped:** load_id, city names, raw date.

4. Models

4.1 Statistical baselines

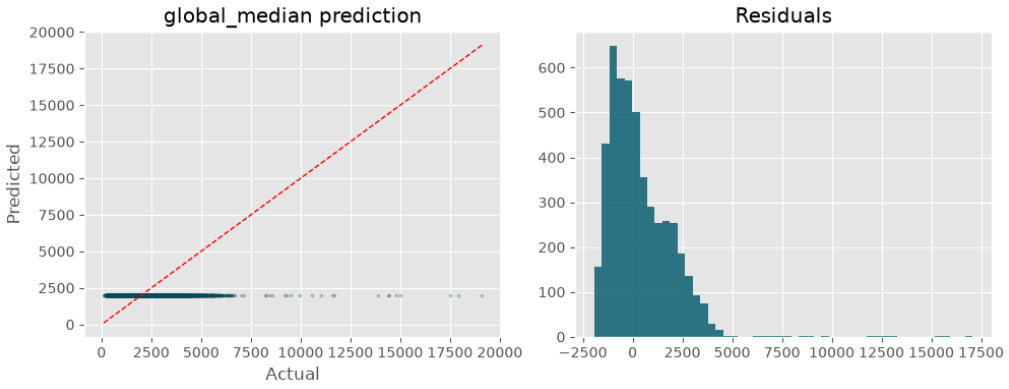
Before using ML I built four simple statistical baselines to set the bar. Each was learned on the training split and scored on the October holdout:

Baseline	RMSE	R2	Idea
Global median	1567.97	-0.052	One value for every load

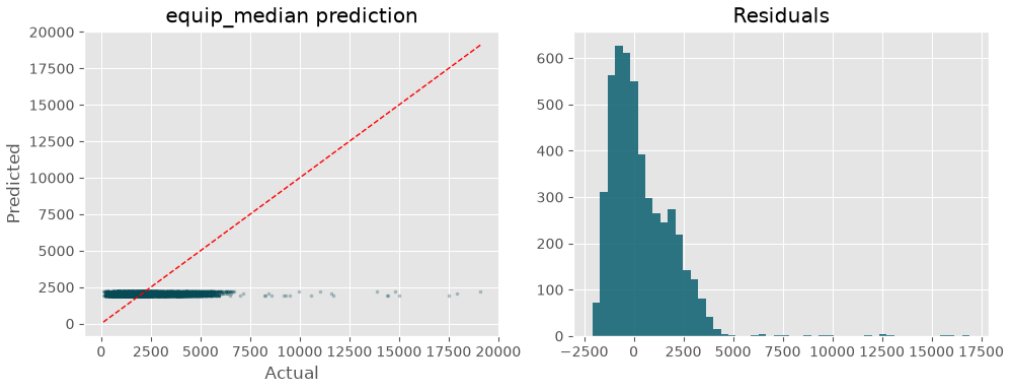
Equipment median	1558.82	-0.040	Median rate per equipment
Equipment rate per mile	683.87	0.800	Median rate/mile by equipment, times distance
Recent 30-day mean	1528.80	-0.000	Mean of the last 30 training days

I noticed that the constant predictors cannot beat R^2 of 0, because rate depends on distance. Multiplying a median rate per mile by distance jumps R^2 to 0.80. So the real benchmark for any ML model is **RMSE about 684**.

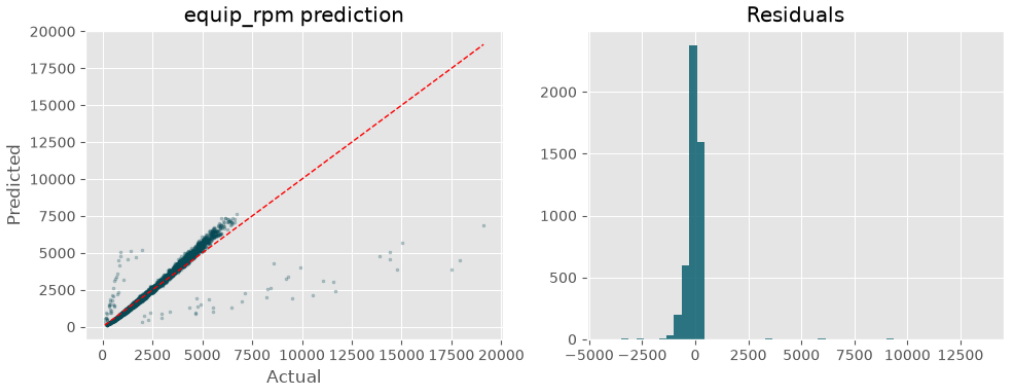
```
rpm = ytrain["posted_rate"] / Xtrain["distance"]
equip_median_rpm = rpm.groupby(Xtrain["equipment"]).median()
pred = Xval["equipment"].map(equip_median_rpm) * Xval["distance"]
```



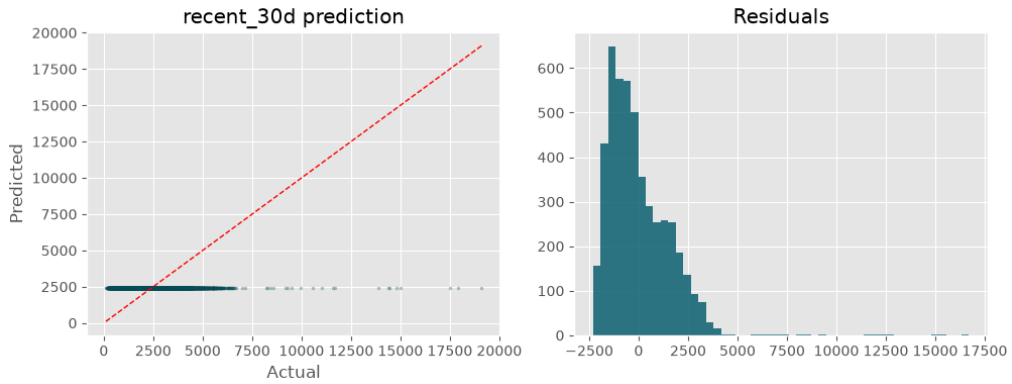
Baseline: global_median (predicted vs actual, residuals).



Baseline: equip_median (predicted vs actual, residuals).



Baseline: equip_rpm (predicted vs actual, residuals).



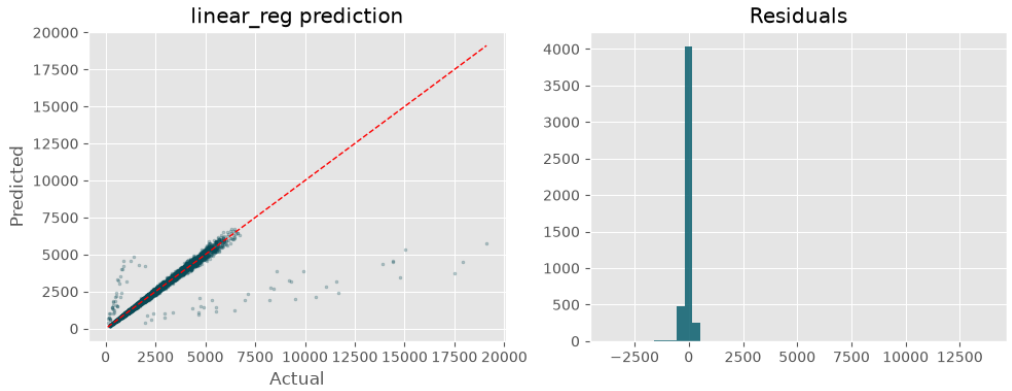
Baseline: recent_30d (predicted vs actual, residuals).

4.2 ML baselines (default parameters, log target)

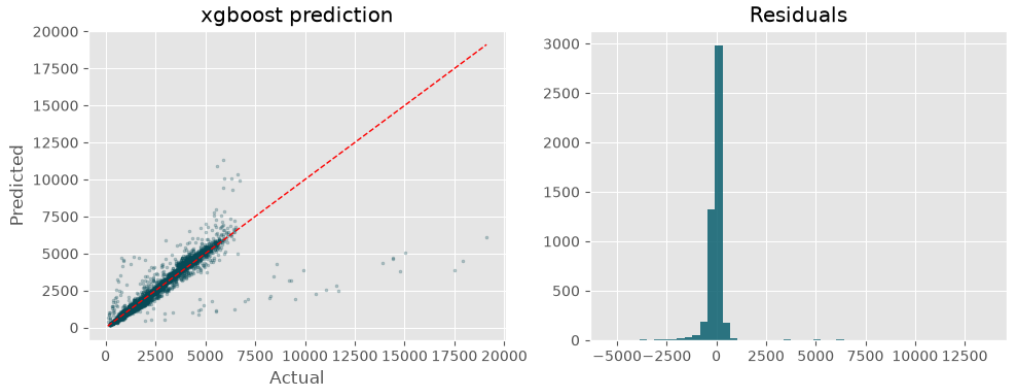
Model	RMSE	R2
Linear regression (scaled)	654.34	0.8168
XGBoost	717.87	0.7794
LightGBM	651.17	0.8185
CatBoost	655.04	0.8164

Next I trained four ML models with default settings on the log target. All of them beat the best statistical baseline except XGBoost. LightGBM scored best out of the box, with CatBoost and plain linear regression close behind. The near tie with linear regression suggests that most of the signal is a simple relationship with distance and the trees add a small amount on top. I picked LightGBM for tuning because it had the best baseline score and trains quickly.

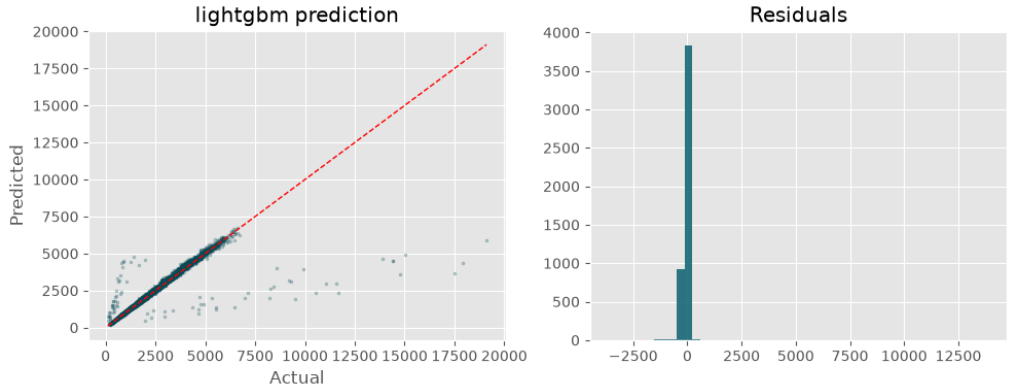
```
def logy(model):
    return TransformedTargetRegressor(regressor=model, func=np.log1p, inverse_func=np.expm1)
models = {"linear_reg": logy(...), "xgboost": logy(XGBRegressor()), "lightgbm": logy(LGBMRegressor()), "catboost": logy(C
```



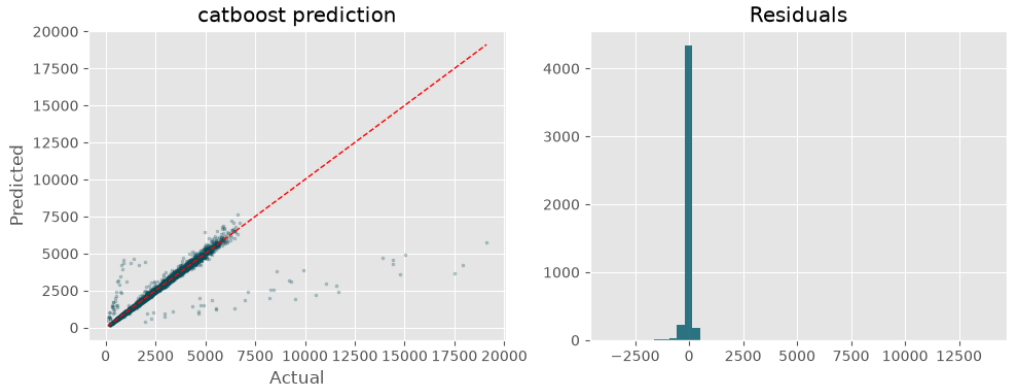
ML baseline: linear_reg.



ML baseline: xgboost.



ML baseline: lightgbm.



ML baseline: catboost.

4.3 Hyperparameter tuning (Optuna)

I tuned LightGBM with Optuna (TPE sampler, median pruner), searching n_estimators, learning_rate, subsample, max_depth, colsample_bytree, L1/L2 regularisation, num_leaves and min_child_samples. I could only run **5 trials** because of time restrictions, as each trial took several minutes across 5 time-series folds. The best trial reached a mean CV RMSE of 415.66. The best parameters:

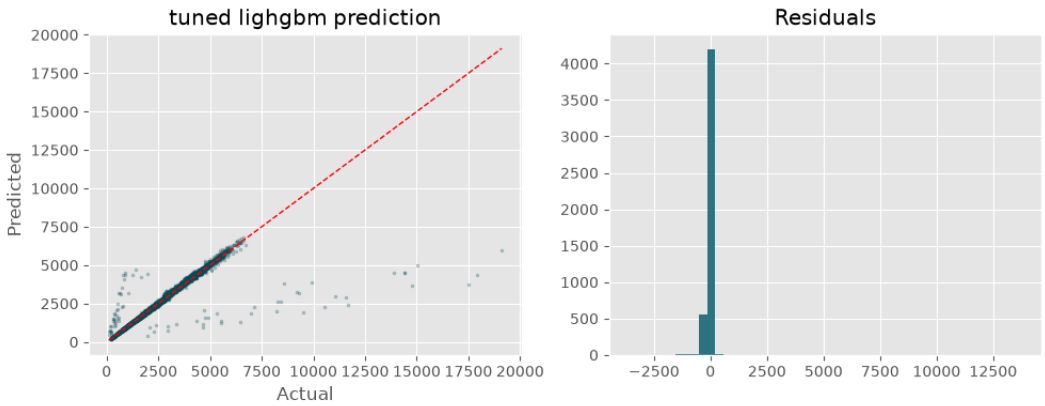
Parameter	Value
n_estimators	600
learning_rate	0.0134
max_depth	6

num_leaves	234
subsample	0.874
colsample_bytree	0.561
reg_alpha	0.0957
reg_lambda	0.00137
min_child_samples	29

The CV RMSE (416) cannot be compared directly with the holdout RMSE (about 650), because the CV folds cover earlier months with a different market level. I only use holdout numbers to compare models.

Stage	Holdout RMSE	Holdout R2
LightGBM default	651.17	0.8185
LightGBM tuned	648.67	0.8199

Tuning improved RMSE by only 2.5 (about 0.4%) over the default. This is small, which is plausible with only 5 trials. The remaining error is probably noise that the available features cannot explain.



Tuned LightGBM on the October holdout.

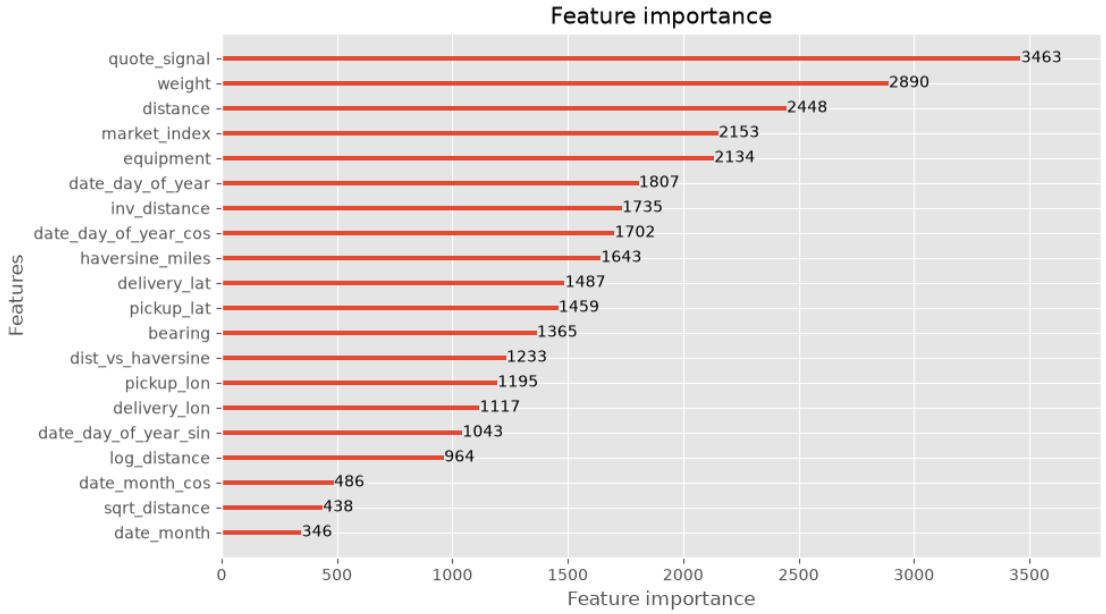
4.4 All models compared (October holdout)

Model	Type	RMSE	R2
Global median	Statistical	1567.97	-0.052
Equipment median	Statistical	1558.82	-0.040
Recent 30-day mean	Statistical	1528.80	-0.000
Equipment rate per mile	Statistical	683.87	0.800
XGBoost	ML baseline	717.87	0.7794
CatBoost	ML baseline	655.04	0.8164
Linear regression	ML baseline	654.34	0.8168
LightGBM default	ML baseline	651.17	0.8185
LightGBM tuned (winner)	**Tuned**	**648.67**	**0.8199**

5. Final model and feature importance

After choosing the parameters, I refitted the feature pipeline and the tuned LightGBM on all 48,000 labeled rows. This final fit has no holdout of its own, so its expected performance is the holdout score above (about 649 RMSE), and it also sees the most recent month of data. I predicted the 12,000 validation loads and wrote them to the template as validation_predictions.csv (load_id, predicted_rate). The model, feature pipeline and weight medians were saved with joblib so predictions can be reproduced without retraining.

```
final_model = build_lgbm(best_params)
final_model.fit(Xtr_full, y_full)
test_template["predicted_rate"] = final_model.predict(Xte)
test_template.to_csv("output/validation_predictions.csv", index=False)
```



LightGBM feature importance (number of splits).

Importance here counts how often a feature is used in splits. **quote_signal**, **weight**, **distance**, **market_index** and **equipment** rank highest. Calendar and geometry features are used moderately, and month-level features are least used. Split counts favour continuous features and do not measure the effect size, so they should be read as a rough guide. It is notable that quote_signal and weight rank above distance in usage even though their raw correlation with rate was near zero, which suggests they act through interactions (for example per-mile price adjustments).

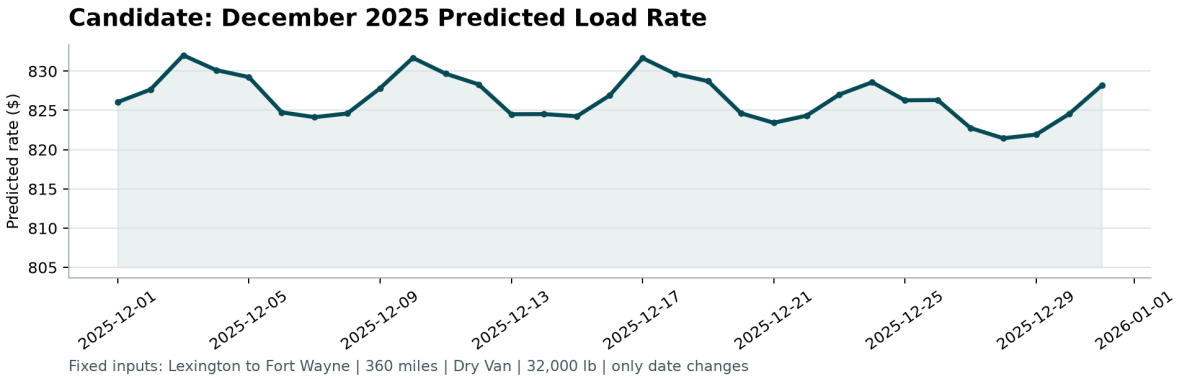
6. December prediction chart

The December input has only 6 usable columns (pickup, delivery, distance, equipment, weight, date: Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb, one row per day). The model needs more than that, so I rebuilt the missing inputs in the same way the model saw them in training:

- **Coordinates:** pickup and delivery lat/lon looked up from the training data by city name.
- **quote_signal:** the median quote_signal across validation loads on each date.
- **market_index:** not available, so it was set to missing and the pipeline filled it with the training median. **load_id:** left empty (unused).
- The columns were aligned to the training schema, passed through the same fitted feature pipeline, and predicted with the final model.

Then I ran the scorer. It validated 12,000 final predictions and 31 December predictions and created scorer_results/candidate_december.png, shown below.

```
python score.py --predictions output/validation_predictions.csv \
    --december-predictions output/december-chart-inputs.csv
```



Fixed December prediction chart from score.py.

7. Limitations

- Only 5 tuning trials were run because of time restrictions, and the gain over the default was small.
- December market_index is imputed with a training median, while the market level has been drifting lower, so December predictions may be biased.
- Validation includes cities and latitudes not seen in training, so some extrapolation is unavoidable.
- Final validation metrics are computed by Spotter after submission, so the true test score is not known.